

FPGA Knight Rider System - Complete Guide

What This System Does

Imagine you have a security system where:

- You enter two numbers using switches
- The system checks if these numbers create the right "password" when combined
- If correct, it shows a cool Knight Rider LED animation (like the car from the 80s TV show)
- If wrong, all LEDs stay off

System Overview

The system has 3 main parts:

1. **Top Module** - The main controller
 2. **Clock Divider** - Makes time go slower for human eyes
 3. **Knight Rider** - Creates the moving LED pattern
-

Part 1: Clock Divider (Making Time Human-Friendly)

The Problem

Your FPGA runs at 100 MHz = 100,000,000 times per second. That's WAY too fast for human eyes!

The Solution

```
verilog
```

```
// If we want 8 Hz (8 times per second):
```

```
count_max = 100,000,000 ÷ 8 = 12,500,000
```

How It Works Step by Step

1. **Counter Setup:** Start counting from 0
2. **Count Up:** Every clock cycle, add 1 to counter
3. **Check:** When counter reaches 12,499,999 (almost at max)
4. **Pulse:** Send out a pulse signal for exactly 1 clock cycle
5. **Reset:** Set counter back to 0
6. **Repeat:** Start counting again

Visual Timeline

verilog

```
if (direction == 0) begin           // Moving RIGHT
    if (position == 13) begin       // Hit the right wall?
        direction <= 1;            // Change to LEFT
        position <= position - 1;   // Move left one step
    end else begin
        position <= position + 1;   // Keep moving right
    end
end else if (direction == 1) begin  // Moving LEFT
    if (position == 0) begin        // Hit the left wall?
        direction <= 0;            // Change to RIGHT
        position <= position + 1;   // Move right one step
    end else begin
        position <= position - 1;   // Keep moving left
    end
end
end
```

Step 3: Light Up 3 LEDs

verilog

```
led_pattern[position +: 3] <= 3'b111; // Turn on 3 LEDs starting at 'position'
```

Why Position Goes 0-13 (Not 0-15)?

- We have 16 LEDs (positions 0-15)
- We light up 3 LEDs at once
- If position = 14, we'd try to light LEDs 14, 15, 16 (but LED 16 doesn't exist!)
- So maximum position = 13 (lights LEDs 13, 14, 15)

Part 3: Top Module (The Main Controller)

Input Processing

verilog

```
wire [7:0] A = sw[7:0]; // First 8 switches = number A
wire [7:0] B = sw[15:8]; // Last 8 switches = number B
```

Password Check

verilog

```
reg [7:0] password = 8'b00001111; // Password = 15 in decimal
```

The system checks: `A XOR B == password`

XOR Operation Example

If A = 10110011 and B = 10111100

A XOR B = 00001111 (which equals our password!)

Main Control Logic

verilog

```
always @(posedge clk) begin
    if (btnC) begin                // Button pressed?
        if ((A ^ B) == password) begin // Password correct?
            led <= led_pattern;      // Show Knight Rider
        end else begin
            led <= 16'b0;             // Wrong password = no LEDs
        end
    end else begin
        led <= 16'b0;               // No button = no LEDs
    end
end
```

Complete System Flow

1. **User sets switches** → A and B values
2. **User presses button** → System activates
3. **System calculates** → A XOR B
4. **Password check** → Compare with stored password
5. **If correct:**
 - Clock divider creates 8Hz pulses
 - Knight Rider updates LED pattern every pulse
 - LEDs show moving animation
6. **If wrong** → All LEDs stay off

Key Programming Concepts

1. Synchronous Design

Everything happens on clock edges - no race conditions!

2. State Machines

Knight Rider tracks position and direction as "state"

3. Clock Domain Management

Different parts run at different speeds (100MHz vs 8Hz)

4. Combinational vs Sequential Logic

- Combinational: XOR, assignments with `assign`
- Sequential: Registers updated with `always @(posedge clk)`

5. Bit Manipulation

- XOR for password checking
 - Bit slicing: `sw[7:0]`, `led_pattern[position +: 3]`
-



What You Can Learn From This

1. **Clock Management** - Essential for any digital system
2. **State Machines** - Used in everything from CPUs to game logic
3. **Real-time Systems** - Managing timing and responsiveness
4. **Security Logic** - Password checking and access control
5. **User Interface** - Switches, buttons, LEDs as I/O

This is a perfect example of how FPGAs let you build custom hardware that does exactly what you want, when you want it!